

6 – EXERCISES ON FUNCTIONS

Quick reference

Listing 1:

```
// A function signature: return_type name(parameter_type parameter, ...)
int clamp(int value, int low, int high); // declaration (prototype)

// Definition
int clamp(int value, int low, int high) {
    if (value < low) return low;
    if (value > high) return high;
    return value;
}

// Overloading: same name, different parameter list
float clamp(float value, float low, float high) { ... }
void clamp(int& value, int low, int high) { ... }
```

Key vocabulary:

- **Signature** — the name plus the list of parameter types (the return type is *not* part of the signature).
- **Return type** — the type written before the function name; `void` means the function returns nothing.
- **Overload** — two functions with the same name but different signatures; the compiler picks the right one based on the arguments passed at the call site.
- **const reference** — `const int& x` gives read-only access to the caller's variable (no copy, no modification).

Part A — Starting with functions

Problem 1 — Calling functions

Jump into this Compiler Explorer session: <https://godbolt.org/z/oxK9jf3or> to complete the exercise.

Complete the `switch-case` so that the correct `print_...` function is called depending on the value of the variable `genre`. Changing the value of `genre` should change the output of the program accordingly.

The solution here: <https://godbolt.org/z/onjr6zrET>.

Problem 2 — Writing functions

Building on the previous solution, write a function `void print_genre(Genre genre)` that prints the music genre of the variable passed as argument. Use that function to simplify your previous main function.

The solution here: <https://godbolt.org/z/qT75Pbdcn>.

Part B — Arguments and references

Problem 1 — Pass by value vs. pass by reference

What does the program in LISTING 2 print?

Listing 2: Value vs. reference

```
1  #include <iostream>
2
3  void boost_v1(int volume, int amount) {
4      volume = volume + amount;
5  }
6
7  void boost_v2(int& volume, int amount) {
8      volume = volume + amount;
9  }
10
11 int main() {
12     int vol1 = 50;
13     int vol2 = 50;
14
15     boost_v1(vol1, 20);
16     boost_v2 (vol2, 20);
17
18     std::cout << vol1 << std::endl; // line A
19     std::cout << vol2 << std::endl; // line B
20     return 0;
```

Line A:

Line B:

Why are the two results different?

Problem 2 — `const` reference

Does the program in LISTING 3 compile? If not, identify the line that causes the error and explain why.

Does it compile?

If not, why?

Listing 3: Const reference

```

1  #include <iostream>
2
3  void print_brightness(const int& value) {
4      std::cout << value << std::endl;
5      value = 100;
6  }
7  int main() {
8      int b = 200;
9      print_brightness(b);
10     return 0;
11 }
```

Problem 3 — Write your own

Jump in this session: <https://godbolt.org/z/3jW6hfedq>.

Inspiring yourself from the function `sum`, complete the function `average` that computes the average of two floating numbers and stores the result in a variable given as the first argument.

Part C — Reading signatures

Problem 1 — Identify the parts

For each function below, fill in the table.

Listing 4:

```
bool is_muted(int volume);
```

```

void set_volume(int& volume, int target);
int mix(int track_a, int track_b);
float pan(float signal, float amount);
void print_genre(const std::string& name);
int clamp_brightness(int value, int lo, int hi);

```

Function	Return type	Parameter types	Returns a value?
is_muted			
set_volume			
mix			
pan			
print_genre			
clamp_brightness			

Problem 2 — Match the call to the signature

The four overloads below all exist at the same time.

Listing 5: Four overloads of describe

```

1 // (A)
2 void describe(int level);
3 // (B)
4 void describe(int level, bool verbose);
5 // (C)
6 void describe(float level);
7 // (D)
8 void describe(const int& level);

```

In TABLE 1, for each call of the first column, write the letter of the overload selected by the compiler, and the return type of that call.

Note: an undecorated integer literal such as 80 is of type `int`. The compiler prefers an exact match over a `const` reference when both are available.

Table 1: Answers

Call	Overload chosen	Return type
<code>describe(80, false)</code>		
<code>describe(80)</code>		
<code>describe(0.5f)</code>		
<code>describe(80, true)</code>		

Part D — Return values

Problem 1 — Trace the return value

What does `main` print in LISTING 6?

Line A:

Line B:

Line C:

Line D:

Listing 6: Note classifier

```

1  #include <iostream>
2
3  int note_value(int note_number) {
4      if (note_number < 1 || note_number > 7)
5          return -1;
6      return note_number * 10;
7  }
8
9  bool is_valid(int note_number) {
10     return note_number >= 1 && note_number <= 7;
11 }
12
13 int main() {
14     std::cout << note_value(3) << std::endl; // line A
15     std::cout << note_value(9) << std::endl; // line B
16     std::cout << is_valid(5) << std::endl; // line C
17     std::cout << is_valid(0) << std::endl; // line D
18     return 0;
19 }
```

Part E — Overloading

Problem 1 — Overload by argument type

Here is a piece of code:

Listing 7: Overloaded `invert`

```
1  #include <iostream>
2
3  int  invert(int value) { return 255 - value; }
4  float invert(float value) { return 1.0f - value; }
5  bool invert(bool value) { return !value; }
6
7  int main() {
8      std::cout << invert(200) << std::endl; // line A
9      std::cout << invert(0.3f) << std::endl; // line B
10     std::cout << invert(true) << std::endl; // line C
11     std::cout << invert(invert(60)) << std::endl; // line D
12     return 0;
13 }
```

For each line A, B, C and D, complete TABLE 2 with the chosen overload and the value returned by `invert`.

Table 2: Answers

Line	Overload chosen	Returned Value
Line A		
Line B		
Line C		
Line D		

Problem 2 — Overload by argument count

Here is a piece of code:

Listing 8: Overloaded `mix`

```
1  #include <iostream>
2
3  int mix(int a, int b) {
4      return (a + b) / 2;
5  }
```

```

5  }
6
7  int mix(int a, int b, int c) {
8      return (a + b + c) / 3;
9  }
10
11 int mix(int a, int b, int c, int d) {
12     return (a + b + c + d) / 4;
13 }
14
15 int main() {
16     std::cout << mix(40, 60)          << std::endl; // line A
17     std::cout << mix(10, 20, 30)     << std::endl; // line B
18     std::cout << mix(0, 100, 50, 150) << std::endl; // line C
19     std::cout << mix(mix(0, 100), mix(50, 150)) << std::endl; // line D
20     return 0;
21 }

```

Complete TABLE 2 with the value printed on each line A, B, C and D.

Table 3: Answers

Line	Printed Value
Line A	
Line B	
Line C	
Line D	

Problem 3 — Overload by `const` reference

Listing 9: Overload on constness

```
1  #include <iostream>
2
3  void inspect(int& value) {
4      std::cout << "non-const: " << value << std::endl;
5      value = value + 1;
6  }
7
8  void inspect(const int& value) {
9      std::cout << "const: " << value << std::endl;
10 }
11
12 int main() {
13     int      a = 80;
14     const int b = 120;
15
16     inspect(a); // line A
17     inspect(b); // line B
18     inspect(50); // line C
19
20     std::cout << a << std::endl; // line D
21     return 0;
22 }
```

Table 4: Answers

Line	Used Overload	Printed Value
Line A		
Line B		
Line C		
Line D	(no call to inspect)	

Why can't line C call the non-`const` overload?

Part F — Spot the bug

Problem 1 — Three buggy functions

Each function below has one bug. Identify each bug and write a fix. You can use the Godbolt links below to write your fix, and then copy-paste the shared link (Go to Share in the upper right corner).

Listing 10: Bug hunt

```
1 // (1) Should return true when volume is between 0 and 100 inclusive.
2 bool is_valid_volume(int volume) {
3     if (volume >= 0 && volume <= 100)
4         return true;
5 }
6
7 // (2) Should double the brightness of a pixel in place.
8 void double_brightness(const int& value) {
9     value = value * 2;
10 }
11
12 // (3) Should compute the average of two ints as a float.
13 float average(int a, int b) {
14     return a + b / 2;
15 }
```

Bug 1:

Fix 1: <https://godbolt.org/z/ccTYraEPM>

Bug 2:

Fix 2: <https://godbolt.org/z/xqqcdo856>

Bug 3:

Fix 3: <https://godbolt.org/z/ff7exsc6r>

Note: to convert a numeric value to a certain type, you can use `static_cast<target_type>(my_var)`.

Part G — Write the code

Problem 1 — Image utilities (mixing all notions)

Write a program containing the following four functions and a `main` function.

Function 1 — clamp

Signature: `int clamp(const int& value, const int lo, const int hi)`

Returns `lo` if `value < lo`, `hi` if `value > hi`, otherwise returns `value`.

Function 2 — clamp (overload)

Signature: `void clamp(int& value, const int lo, const int hi)`

Same logic as above, but modifies `value` in place and returns nothing.

Function 3 — is_valid_pixel

Signature: `bool is_valid_pixel(int r, int g, int b)`

Returns `true` only if all three values are between 0 and 255 inclusive.

Function 4 — print_pixel

Signature: `void print_pixel(const int& r, const int& g, const int& b)`

Prints the three values on one line separated by spaces.

In `main`:

1. Call `clamp(300, 0, 255)` (value overload) and print the result.
2. Declare `int r = 300, g = 150, b = -10`
 - (a) Call the reference overload of `clamp` on each with `lo=0, hi=255`;
 - (b) Call `print_pixel(r, g, b)`.
3. Print the result of `is_valid_pixel(r, g, b)`.

Expected output:

Listing 11:

```
255
255 150 0
1
```

[Link to your solution :](#)

Problem 2 — Music utilities

Write a program containing the following functions and a `main` that calls each of them.

Function 1 — tempo_label

Signature: `void tempo_label(int bpm)`

Prints a label based on the value of `bpm`:

Function 2 — tempo_label (overload)

Signature: `void tempo_label(int bpm, bool verbose)`

When `verbose` is `true`, prints `"bpm: N -- label"`, where `N` is the `bpm` value and `label` is the label described in TABLE 5. When `verbose` is `false`, behaves exactly like the one-argument version.

Table 5: BPM Labels

Condition	Label
bpm < 60	"slow"
60 <= bpm < 120	"medium"
120 <= bpm < 180	"fast"
otherwise	"very fast"

Hint: you can call the one-argument overload from inside this one.

Function 3 — beats_in_bars

Signature: `int beats_in_bars(int bars, int beats_per_bar)`

Returns the total number of beats in `bars` bars.

Function 4 — scale_bpm

Signature: `void scale_bpm(int& bpm, float factor)`

Multiplies `bpm` by `factor` in place, storing the result as an `int` (truncation is acceptable).

In `main`:

1. Declare `int bpm = 90`.
2. Call `tempo_label(bpm)`.
3. Call `tempo_label(bpm, true)`.
4. Print `beats_in_bars(4, 4)`.
5. Call `scale_bpm(bpm, 1.5f)` then print `bpm`.
6. Call `tempo_label(bpm, true)`.

Expected output:

Listing 12:

```
medium
bpm: 90 -- medium
16
135
bpm: 135 -- fast
```

Link to your solution :